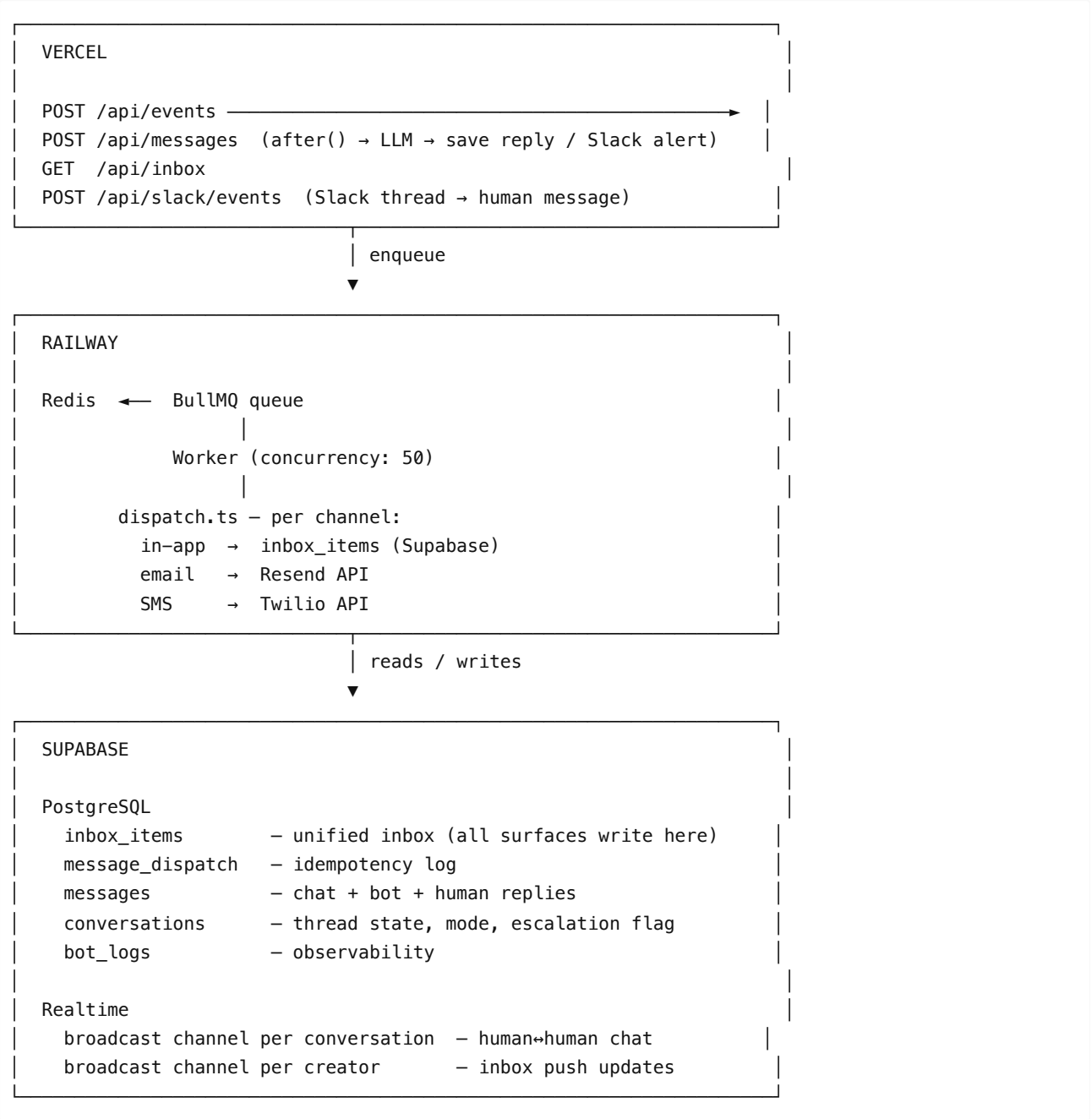
Creator Communication Platform — Engineering Assignment Submission

---

1. Architecture Overview

The four surfaces and how they share infrastructure

The system runs across three platforms. Supabase is the data layer everything reads and writes to. Vercel handles the stateless API surface. Railway runs the long-lived worker that Vercel can't host.



Why this split:

Vercel handles the API routes — stateless, short-lived, scales horizontally. It cannot run the BullMQ worker because Vercel functions terminate after the response; a persistent queue worker needs a process that stays alive. That's Railway.

Supabase owns the data layer and Realtime — both the Vercel routes and the Railway worker read and write to the same Postgres database, which is how they stay in sync without any direct coupling between them.

How the unified inbox materializes

The inbox is a physical `inbox_items` table — not a view, not a query-time union. Every surface writes to it:

- **Automated messages**: `dispatchInApp()` inserts one row per event delivery
- **Support bot**: inserts a row of `type = 'support'` when a conversation starts and when bot/human replies land
- **Human chat**: inserts a row of `type = 'chat'` per new message in a conversation the creator hasn't read

Why a table and not a view? At 5M creators with millions of inbox items, a `UNION ALL` view across messages/payments/posts/etc runs four table scans per page load. The materialized `inbox_items` table with `(creator_id, created_at DESC)` index means every inbox load is a single indexed range scan. The tradeoff: writes are slightly more complex (every source must write to inbox_items). That's manageable. A slow inbox is not.

The boundary between messaging and the rest of the product

The product emits domain events via `POST /api/events` (authenticated with `x-events-secret`). Everything in this system is downstream of that boundary. The payment service doesn't know about Resend. The review pipeline doesn't know about inbox_items. They fire `payment.sent` or `post.rejected` and stop. The communication platform owns the rest.

---

2. Event Pipeline for Automated Messages

How `payment.sent` goes from event to delivery

```
1. Payment service:
   POST /api/events
   { name: "event/payment.sent", data: { paymentId, creatorId, amount, videoCount } }
   → 200 OK (acknowledged, not yet delivered)

2. API route:
   getEventQueue().add("event/payment.sent", data)
   → enqueued in BullMQ/Redis

3. Worker (Railway):
   Picks up job (concurrency: 50)
   fetchCreator(creatorId) → { name, email }
   Build TemplateCtx

4. Dispatch in parallel:
   dispatchInApp()  → claimDispatch("payment.sent:pay_123:in-app")
                    → INSERT INTO message_dispatch (idempotency_key, status)
                    → on success: INSERT INTO inbox_items
                    → markDispatched()

   dispatchEmail()  → claimDispatch("payment.sent:pay_123:email")
                    → check quiet hours
                    → POST https://api.resend.com/emails
                    → markDispatched()
```

The response to the payment service is `200 OK` the moment the event is enqueued. Delivery is async. If the worker crashes mid-job, BullMQ retries with exponential backoff. The idempotency key ensures the retry never double-sends.

Idempotency: how `payment.sent` never double-sends

The `message_dispatch` table has `idempotency_key TEXT PRIMARY KEY`. The `claimDispatch()` function does:

```
INSERT INTO message_dispatch (idempotency_key, status)
VALUES ($1, 'pending')
-- PRIMARY KEY constraint — this fails silently if the key already exists
```

If the INSERT succeeds: this worker owns this delivery. Proceed. If the INSERT fails (duplicate key): another worker already claimed it. Return early.

This is a distributed lock, not a flag check. There's no TOCTOU race because the claim and the uniqueness check are the same atomic operation. BullMQ can retry the job 50 times, Vercel can redeploy mid-job, two workers can race — exactly one delivery happens.

For `payment.sent` specifically, the idempotency key is `"payment.sent:{paymentId}:email"`. The `paymentId` is the stable entity ID from the source — not a timestamp, not a UUID generated at dispatch time. The key survives retries and redeploys.

Fanout: 10,000 `creator.dropped` events in one minute

When a campaign ends, the product fires a single `event/campaign.ended` event (not 10,000 individual ones). The worker handles it:

```
case "event/campaign.ended": {
  const creatorIds = await fetchAllCreatorsOnCampaign(campaignId) // single query
  await getEventQueue().addBulk(
    creatorIds.map(id => ({
      name: "event/creator.dropped",
      data: { creatorId: id, campaignId, brandName }
    }))
  )
  break
}
```

`addBulk()` pushes all 10,000 jobs into Redis in a single pipeline. The queue acts as the buffer. Workers pull from it at `concurrency: 50` — that's 50 jobs processed in parallel. At that rate, 10,000 jobs complete in ~3–4 minutes, not in one burst.

The email provider (Resend) has rate limits. The queue naturally throttles this — 50 concurrent workers, each taking ~200ms per email dispatch = ~250 emails/second sustained throughput. Resend's limits are per-account; if they're lower, we add a rate-limit wrapper or use Resend's batch endpoint.

The critical insight: **the cron or the product trigger never directly calls Resend or writes to inbox_items**. Everything goes through the queue. The queue is the backpressure valve.

Template system

Templates live in `lib/templates/index.ts`. Each event type has a `Template` object:

```
interface Template {
  channels: Channel[]          // which channels this event uses
  subject: (ctx) => string     // email subject
  preview: (ctx) => string     // inbox preview text (one line)
  emailBody: (ctx) => string   // email body
  smsBody?: (ctx) => string    // SMS text (only for job.offered)
}
```

The `TemplateCtx` is a flat key-value map populated by the worker after fetching the relevant entities. Templates are code, not database rows. This is intentional: templates change with deploys, not data migrations. Previewing is done in tests — the functions are pure, so `template.preview({ creatorName: "Jordan", amount: 24000, videoCount: 3 })` works in any test runner.

Localization: the model can generate responses in the creator's language (see Section 3). For system templates, we add a `locale` field to creators and branch in the template function. Not wired up yet — deferred until there's a non-English creator base large enough to justify the QA overhead.

---

3. FAQ / Support Bot

What triggers the bot vs. the human queue

Every inbound creator message goes through the bot first. The bot classifies intent using a forced tool call (`support_response`) and routes accordingly:

Intent	Condition	Action
`DATA`	Answerable from creator's verified DB record	Bot replies, logs to bot_logs
`GENERAL`	Static how-to (warmup, Spark Codes, Stripe)	Bot replies, logs to bot_logs
`ESCALATE`	Payment disputes, drop risk, anger, missing data, judgment calls	Bot sends warm handoff, fires Slack alert, tags conversation escalated

The model never chooses freely. The system prompt contains explicit boolean rules, not soft guidelines:

```
ESCALATION RULES (absolute, no exceptions):
- Creator mentions not being paid, disputes any amount → ESCALATE
- Creator asks if they'll be dropped or removed → ESCALATE
- Creator wants to quit or leave → ESCALATE
- Any required data field is NOT SET and creator asks about it → ESCALATE
- Creator appears angry or upset → ESCALATE
```

False positives (escalating something the bot could have answered) cost a few seconds of staff time. False negatives (bot guessing wrong on a payment dispute) cost creator trust. The rules are biased accordingly.

How context is fetched and injected

`getContext(creatorId, campaignId?)` runs three indexed queries in the background after the 202 is returned:

```sql
SELECT * FROM creators WHERE id = $1;
SELECT campaign_id FROM creator_campaigns WHERE creator_id = $1 AND active = true;
SELECT * FROM campaigns WHERE id = $campaignId;
```

All fields are serialized into the system prompt as labeled sections. NULL fields are written as `"NOT SET"` explicitly — the model sees the absence, not a gap. The DECISION RULES section instructs the model: if a field is NOT SET and the creator asks about it, escalate without explaining why.

This prevents hallucination structurally. The model cannot invent a pay rate because the pay rate is either in the context or it's `NOT SET → ESCALATE`. There is no third path.

How 8x staff take over a bot-handled thread

Human handoff works entirely through Slack — no separate admin dashboard needed for the current scale.

```
1. Bot escalates → chat.postMessage to #support
   → Slack returns message ts
   → stored as conversations.slack_thread_ts

2. Staff member replies in that Slack thread
   → Slack Events API fires POST /api/slack/events
   → look up conversation by slack_thread_ts
   → saveMessage({ role: "human", content: "..." })
   → conversations.mode = 'human'

3. Creator sees reply in their chat UI via polling
   — they never know it came from Slack
```

Once `conversations.mode = 'human'`, the bot no longer processes new messages in that thread. The mode column is the handoff boundary. A staff member can also set mode back to 'bot' if the issue is resolved and the creator has further routine questions.

Measuring bot correctness without reading every conversation

The `bot_logs` table records every `(creator_id, message, bot_response, escalated, created_at)` tuple. Three signals:

1. Escalation rate — what % of messages escalate vs. self-resolve. A healthy bot escalates 20–30% of messages. If it's 60%, the rules are too conservative. If it's 5%, something is wrong.

```sql
SELECT escalated, count(*) FROM bot_logs
WHERE created_at > now() - interval '7 days'
GROUP BY escalated;
```

2. Thumbs-down rate — after every DATA or GENERAL reply, the creator sees thumbs up/down. `helpful = false` on the `bot_logs` row. This is the direct signal for which answers need the system prompt tightened.

3. Spot audit — periodic review of `escalated = false` bot answers. This catches systematic errors the thumbs-down doesn't surface (creators who don't bother rating, or are satisfied with a wrong answer but act on it incorrectly).

---

4. Human ↔ Human Messaging

How to fix the realtime fanout / RLS-leak problem

The current system subscribes every authenticated client to every `messages` INSERT in the database and filters client-side. At 100k concurrent creators, this means every single message write broadcasts to every connected client. That's catastrophically wrong: it leaks metadata (any creator can see who else is messaging) and saturates every connection with irrelevant events.

The fix: per-conversation Supabase Realtime broadcast channels.

Instead of subscribing to the `messages` table directly, each client subscribes to a broadcast channel named after the conversation:

```ts
// Client
const channel = supabase.channel(`conversation:${conversationId}`)
channel.on('broadcast', { event: 'new_message' }, ({ payload }) => {
  appendMessage(payload)
})
.subscribe()
```

When the server saves a message, it broadcasts to that channel:

```
// Server (route handler, after INSERT)
await supabase.channel(`conversation:${conversationId}`)
  .send({ type: 'broadcast', event: 'new_message', payload: message })
```

Each client only receives messages for conversations they're subscribed to. The RLS leak is gone — Supabase Realtime broadcast channels are ephemeral; they don't touch the database at all, so there's no row-level security question. The server is the authority on who can subscribe (it validates the JWT before writing the message and before broadcasting).

This also fixes the fanout problem. A message in conversation A triggers one broadcast to subscribers of `conversation:A`. Creators in 100k other conversations don't receive anything.

Keeping it scalable at 100k concurrent creators

Supabase Realtime is built on Phoenix Channels and can handle hundreds of thousands of concurrent WebSocket connections across a cluster. The per-conversation channel model means each connection only carries traffic relevant to that user.

What actually breaks at scale isn't Realtime — it's the `messages` table. At 5M creators with thousands of messages per day, even indexed queries on `(conversation_id, created_at)` start to slow down on a single Postgres instance. The mitigation: partition `messages` by `created_at` (monthly), and move conversations older than 90 days to cold storage (S3 + a `message_archives` pointer table). The hot table stays small.

Read receipts, typing indicators, attachments

Typing indicators: ephemeral — sent as Realtime broadcast events, never persisted. The client sends `{ event: 'typing', payload: { conversationId, userId } }` on keystroke (debounced 1s). Other subscribers see it and clear it after 3s if no follow-up. Zero DB writes.

Read receipts: `messages.read_at` column, set by `POST /api/messages/{id}/read`. Also reflected in `inbox_items.read_at`. The client sends a read receipt when the conversation view mounts and when it receives new messages while focused. Batched — one write per conversation open, not per message.

File attachments: upload directly to Supabase Storage via a presigned URL; the resulting storage path is saved as `messages.attachment_url`. The message row is created after upload completes. This avoids sending binary data through the API route and keeps message creation atomic — either the file is uploaded and the message exists, or neither does.

All three are in scope for the first real ship. Typing and read receipts add almost no cost and are table-stakes UX for a chat product.

---

5. Unified Inbox

Data model

```
CREATE TABLE inbox_items (
  id           UUID PRIMARY KEY,
  creator_id   UUID NOT NULL REFERENCES creators(id),
  type         TEXT CHECK (type IN ('system', 'chat', 'support')),
  thread_id    UUID,          -- conversation_id for chat/support rows
  preview      TEXT,          -- one-line display text
  entity_type  TEXT,          -- 'payment' | 'post' | 'campaign' | 'job'
  entity_id    UUID,          -- deep-link target
  read_at      TIMESTAMPTZ,
  metadata     JSONB,         -- { idempotency_key, event_type, source_id }
```

```
  created_at   TIMESTAMPTZ
);

-- Fast inbox load
CREATE INDEX idx_inbox_creator_created ON inbox_items (creator_id, created_at DESC);

-- Fast unread badge count
CREATE INDEX idx_inbox_unread ON inbox_items (creator_id) WHERE read_at IS NULL;

-- Idempotency: one inbox row per event+channel per creator
CREATE UNIQUE INDEX idx_inbox_idempotency
  ON inbox_items (creator_id, (metadata->>'idempotency_key'))
  WHERE metadata->>'idempotency_key' IS NOT NULL;
```

Every surface writes here. `type = 'system'` is an automated event message. `type = 'support'` is a support conversation update. `type = 'chat'` is a brand/staff chat update. One table, one index, one query for the inbox feed.

How the client subscribes to updates

The client does two things on mount:

1. `GET /api/inbox?creatorId=xxx&limit=50` — initial load, cursor-paginated
2. Subscribe to Supabase Realtime broadcast on `inbox:{creatorId}` — receives new rows as they land

When the server inserts into `inbox_items`, it also broadcasts:

```
await supabase.channel(`inbox:${creatorId}`)
  .send({ type: 'broadcast', event: 'new_item', payload: inboxItem })
```

The client appends the new item to the top of the feed. No polling. No full re-fetch.

This is better than subscribing to the `inbox_items` table directly (which would trigger on every INSERT across all creators and require RLS to filter). The broadcast is targeted by construction.

Read state, quiet hours, digest rollups

Read state: `inbox_items.read_at` is set by `POST /api/inbox/{id}/read`. The client calls this when the creator taps a row. Unread count = `WHERE creator_id = $1 AND read_at IS NULL`. The inbox badge queries this index.

Quiet hours: `creators.quiet_hours_start`, `creators.quiet_hours_end`, `creators.timezone`. The `isQuietHours()` utility checks these before dispatching email or SMS. In-app notifications are not suppressed during quiet hours — they're silent (no push). Push is suppressed. The creator configured the quiet window; in-app delivery during that window is fine since they're presumably asleep and the notification won't interrupt them.

Digest rollups: `post.approved` is high-volume (thousands/day at scale). Rather than flooding the inbox with 50 individual "Your post was approved" rows, the worker checks if there's already an unread `post.approved` row from the same day for this creator:

```
// Before inserting a new post.approved inbox row:
const existing = await supabase
  .from("inbox_items")
  .select("id, metadata")
  .eq("creator_id", creatorId)
  .eq("type", "system")
  .contains("metadata", { event_type: "post.approved" })
  .gte("created_at", startOfDay)
  .is("read_at", null)
```

```
  .single()

if (existing) {
  // Update the existing row's preview and increment count
  await supabase.from("inbox_items").update({
    preview: `${existing.metadata.approvedCount + 1} posts approved today`,
    metadata: { ...existing.metadata, approvedCount: existing.metadata.approvedCount + 1 }
  }).eq("id", existing.id)
} else {
  // Insert new row
}
```

This keeps the inbox readable. A creator who had 50 posts approved today sees one row: "50 posts approved today", not 50 rows. `payment.sent`, `post.rejected`, `creator.dropped` are never rolled up — each is significant and deserves its own row.

---

6. Trade-offs & Failure Modes

Where I picked "fast to ship" over "right long-term"

`after()` instead of a proper queue for the bot: At 50–100 support messages/day, `next/server after()` is fine. The LLM call runs post-response, the creator waits 1–3 seconds, and there's no infra overhead. At 50,000 messages/day, `after()` becomes a liability — Vercel function cold starts and timeout limits make it unreliable. The fix is a queue (Inngest or BullMQ) + a dedicated worker, same pattern as the event pipeline. I've kept the architecture parallel so this migration is a swap, not a redesign.

Slack as the human handoff interface: A proper support dashboard (agent assignment, SLA tracking, conversation routing) is the right long-term answer. Slack works at current scale (a few dozen escalations/day). At 1,000 escalations/day, a dedicated dashboard becomes necessary. The `conversations.mode` and `escalated` columns are already the right data model for this.

Polling for bot replies instead of WebSockets: The creator waits up to the poll interval (2s) for the reply. WebSockets would eliminate this lag. Polling is simpler to reason about and deploy, and the latency difference (0–2s) is acceptable. The switch to WebSockets is a client-only change — the server API contract doesn't need to change.

Where I picked "right long-term" over "fast to ship"

Idempotency from day one: I could have shipped without the `message_dispatch` table and just checked for existing inbox rows before inserting. I didn't, because that check has a race condition — two workers can both check, both see nothing, both insert. The `claimDispatch()` pattern using a PRIMARY KEY INSERT is correct and the table is tiny. There's no good reason to skip it.

Physical `inbox_items` table instead of a view: A `UNION ALL` view is the "fast to ship" choice — no extra writes, query-time materialization. At 5M creators with millions of daily events, it becomes untenable. The physical table is slightly more complex to maintain but doesn't fall over at scale.

Templates as code, not database rows: Database-managed templates seem flexible but create a deployment dependency (template changes require a data migration or an admin UI). Code templates ship with the deploy, are testable, and are diffs in version control. The "flexibility" of database templates isn't worth the operational cost for a team this size.

What breaks at 500k creators? At 5M?

At 500k creators:

- The `messages` table starts getting large (assume 5 messages/day/creator = 2.5M rows/day). Queries slow without partitioning.

- The BullMQ worker at `concurrency: 50` handles ~250 emails/second. A 500k-creator campaign end means ~500k `creator.dropped` jobs. At 250/s, that's ~33 minutes to clear the queue. Acceptable, but the email queue depth needs monitoring.
- Supabase connection pooling becomes a concern (PgBouncer is on by default, but the worker and API routes can exhaust it under burst load).

At 5M creators:

- Single Postgres instance hits its limit for write throughput. Need read replicas for the inbox API, and potentially sharding by `creator_id` for the `messages` and `inbox_items` tables.
- The BullMQ worker needs to be a cluster (10+ Railway instances, Redis Cluster). The queue architecture supports this — BullMQ is designed for distributed workers.
- Supabase Realtime at 5M concurrent WebSocket connections is beyond what a single Supabase project handles. At that scale, we'd move to a dedicated Realtime service (Soketi, Ably, or self-hosted Phoenix) and keep Supabase as the data layer only.
- Email cost at 5M creators: Resend charges ~$0.001/email. At 2 emails/creator/day = $10,000/day in email alone. Switch to bulk/transactional tiers with SES or Postmark at volume.
- Bot cost: Claude Haiku at ~$0.001/message (including context). At 5M creators sending 1 message/week = ~$5,000/week. Manageable, but context window size matters — trimming the system prompt and capping history at 5 messages is the right call.

Migration path from the current four-system mess

The migration must be additive — nothing breaks while the new system rolls out. The key principle: **never do a flag day**.

Phase 1 — event pipeline + bot (first two weeks): Deploy the BullMQ worker and `message_dispatch` table alongside the existing ad-hoc triggers. New code emits events to the queue AND fires the old trigger in parallel. Verify idempotency keys prevent double-delivery during overlap. Once confirmed, kill the old trigger one event type at a time — `payment.sent` first, then the rest. Deploy the support bot in parallel; it slots between message storage and Slack notification without touching anything else.

Phase 2 — unified inbox + deprecation: Ship the `inbox_items` table and inbox API. Existing notification emails keep going out while the new inbox rows are created in parallel. Ship the inbox UI once the data is confirmed stable. Fix the Supabase Realtime subscription for human chat (per-conversation broadcast channels — client-side only change). Once every event type has been running through the queue for two weeks with full observability, deprecate the four old systems one by one.

What this costs per month at 5M creators

Baseline (no optimizations):

Component	Cost basis	Monthly
Supabase (Pro+)	Storage, connections, bandwidth	~$500–2,000
Redis + Railway worker	BullMQ queue + worker process	~$300–800
Vercel (Pro)	API route invocations	~$500–2,000
Resend (email)	~40M emails/month (payments, rejections, job offers)	~$40,000
Twilio (SMS)	`job.offered` only, ~5% of creators/day	~$3,000–5,000
Claude Haiku	1 support message/creator/week = 20M msgs/month	~$18,000
Total		**~$62,000–68,000/month**

With cost optimizations applied:

Lever	Saving	How

Prompt caching (Claude)	~60% off LLM cost	The static sections of the system prompt (STATIC INSTRUCTIONS, DECISION RULES) never change between creators. Claude's prompt caching prices cached input tokens at ~$0.08/MTok vs $0.80/MTok — roughly 10× cheaper for the cacheable prefix. Brings Haiku cost to ~$7,000/month.
Switch email provider at volume	~80% off email cost	Resend is priced for low-to-mid volume. At 40M+ emails/month, AWS SES ($0.0001/email) or Postmark's bulk tier cuts the bill to ~$4,000–8,000/month.
In-app only for `post.approved`	Already done	Eliminating email for the highest-volume event alone saves the most.
Push instead of email for time-sensitive events	~30% off remaining email	`warmup.reminder` and `viral.alert` are better as push notifications — cheaper and faster. Push via FCM/APNs costs ~$0.0005/notification.

Optimised total: ~$18,000–25,000/month

Scaling the worker as event volume grows: The Railway worker runs at `concurrency: 50` today — sufficient for current load. As creators and events increase, the worker is horizontally scalable: add more Railway instances pointing at the same Redis queue, each pulling jobs independently. BullMQ distributes work across all connected workers automatically. No code changes required — just spin up more instances and set the concurrency per instance based on available memory. At 5M creators with burst fanouts (10k+ events/minute), running 5–10 worker instances at concurrency 50 each gives ~2,500 jobs/minute throughput, which clears a 10k-creator campaign drop in under 5 minutes.

The cost cliff is email, not the AI. The model cost is a rounding error compared to email at scale. The LLM is also fully swappable — the tool call structure works identically with any model that supports structured outputs (GPT-4.1 mini, Gemini Flash, etc.). Switching providers is a one-line change in `lib/bot/llm.ts` if pricing shifts.

---

7. What I'd Build First

Done

Event pipeline — BullMQ worker on Railway, all 8 event types handled, idempotent dispatch via `message_dispatch` table, templates for in-app / email / SMS, quiet hours respected, fanout via `addBulk()` for `campaign.ended`.

Support bot — `POST /api/messages` with `after()` async LLM pipeline, context injection from creator + campaign records, forced tool-call intent classification (DATA / GENERAL / ESCALATE), Slack escalation alerts, Slack thread sync so human replies flow back to the creator's chat.

Unified inbox — `inbox_items` table, `GET /api/inbox` API, idempotency index, unread count, read state. Full two-column inbox page at `/inbox` (thread list + active thread panel) with per-creator Supabase Realtime broadcast subscription for instant updates. Floating widget on marketing page reuses the same components.

`post.approved` digest rollups — worker checks for an existing unread `post_batch` row today before inserting; updates the count and preview instead of creating N individual rows.

Human chat Realtime fix — inbox subscription switched to per-creator broadcast channel; server broadcasts on every `dispatchInApp()` insert. No table-level subscription, no RLS leak.

Observability — `bot_logs` table, thumbs-down feedback signal, escalation rate queries.

Database — RLS policies on all tables, read receipts on messages, quiet hours on creators, `conversations.mode` for bot/human handoff.

Still to build

Human support dashboard — Slack handles current escalation volume. A proper dashboard with agent assignment, conversation routing, and SLA tracking becomes necessary around 1,000 escalations/day.

---

8. How I Used AI During This Assignment

I used Claude (claude.ai/code) as a coding collaborator throughout — architecture partner, implementation engine, and reviewer.

Where it helped:

- **Scaffolding the event pipeline architecture**: I described the requirement (idempotent fanout, BullMQ, per-event templates) and Claude generated the initial `dispatch.ts`, `worker.ts`, and `templates/index.ts`. The structure was right; I adjusted the `claimDispatch()` implementation to use a PRIMARY KEY INSERT rather than a SELECT-then-INSERT pattern (which has a race condition).
- **Database migrations**: Claude generated all nine migrations from schema descriptions. Faster than writing DDL by hand and less error-prone.
- **The Slack thread sync handoff**: I asked how a human could pick up a bot-escalated conversation through Slack without a separate admin dashboard. Claude designed the full pattern (capture `ts` from `chat.postMessage`, store as `slack_thread_ts`, webhook to map thread replies back to conversation). Went from question to working implementation in one session.
- **Catching a coupling issue**: `formatPayInfo` was exported from `slack.ts` but logically belonged in the route layer. Claude flagged it during a code review pass.

Where I had to override or correct it:

Claude initially suggested a confidence threshold for bot escalation — "escalate if confidence < 0.7." I pushed back: there's no reliable way to get a calibrated confidence score from an LLM, and a 0.7 threshold on payment disputes is far too risky. The correct model is a decision tree with hard boolean rules, not a probability. I rewrote the escalation logic entirely.

Claude suggested a `UNION ALL` view for the unified inbox ("just query messages, inbox_items, and payments at read time"). I rejected this. At 5M creators, a view that unions four tables on every inbox load doesn't scale. The physical `inbox_items` table with a write-time materialization pattern is the right answer, even though it's more work to maintain. Claude's instinct was "simple query now," mine was "fast read at scale."

Claude generated a more complex worker architecture initially with Inngest instead of BullMQ. Inngest is a good product but adds a managed-service dependency and a pricing layer that doesn't justify itself at current volume. BullMQ on a Railway Redis instance is simpler, cheaper, and directly controllable. I switched it.

One moment where I disagreed and was right:

Claude suggested adding full conversation history (all prior messages) to the LLM context for every bot call, reasoning that multi-turn context would improve answer quality. I explicitly chose not to for the initial version: most creator questions are self-contained ("when do I get paid?", "how do I set up Spark Codes?"), and adding conversation history adds token cost and latency on every message. I capped it at 5 messages, not because the model couldn't handle more, but because the cost-to-benefit ratio doesn't justify it at current question complexity. Multi-turn context becomes valuable when creators have ongoing complex discussions — that's a v2 feature, not a v1 requirement.

Time spent

Total: ~14 hours across 3 days.

- Architecture + system design: ~5 hours
- Implementation (migrations, worker, dispatch, bot, routes): ~7 hours
- Writing this document: ~2 hours

Submitted by Shashank Telkhade — shashank.telkhade@gmail.com